There are a few more tricks you can pull with blocks, such as nesting and a conditional form of {% extend %} (which accepts Twig expressions, not just string literal file paths). You can read about them at https://twig.symfony.com/doc/tags/extends.html and https://twig.symfony.com/doc/tags/use.html

Extending Twig in PHP

What if you just cannot achieve what you want using Twig's syntax and features? Well, there's always the nuclear option! It is possible to extend the Twig templating language in PHP: you can add your own global variables, filters, and functions with ease. Whether you should do so is another question; the logic or functionality you want to introduce into the view might be better off elsewhere in the MVC architecture (perhaps as a service or logic in a controller). Generally speaking, you should have everything in order (data fetched, decisions made, environment configured, etc.) before passing control to the view. Ducking out to fetch more data or apply more business logic when you're halfway through assembling your content is a bad idea. Nevertheless, there are occasionally cases where adding extra functionality to your templating engine is appropriate.

The basic principle for either is straightforward: for filters and functions, Twig provides `Twig\TwigFilter` and `Twig\TwigFunction` classes, which can be constructed with a name string and some way to call through to the underlying PHP implementation (anonymous function, static class method, and so on). An optional third argument is an array of options to control behavior (such as HTML escaping). These objects are then registered with the `Twig\Environment` object that will be doing the rendering. Adding a "global" is even easier: any data you like can be exposed to all the templates visited by `::render()` by simply binding a context name to it:

```
$twig->addGlobal('amazingHelper', $object);
```

The code above exposes all of the public properties and methods of `$object` via `amazingHelper`, and they are accessed in the usual way: {{ amazingHelper.property }}, {% amazingHelper.method() %} anywhere in the chain of `extends`, `use`, or `include` calls.